Requirements Gathering

1. Stakeholder Analysis

Identifying the key stakeholders and their needs is crucial for the success of the predictive maintenance system. The primary stakeholders include:

· **Plant Managers & Executives:** Require high-level dashboards and ROI metrics (e.g., reduced downtime, cost savings) to justify the investment and monitor overall operational efficiency.

· **Maintenance Engineers & Technicians:** Need actionable alerts, failure probabilities, and specific sensor anomaly reports to schedule timely repairs and prevent catastrophic equipment failure.

· **Machine Operators:** Need a non-intrusive system that provides real-time health status of the machinery they operate to ensure safety and continuous workflow.

· **Data Scientists & ML Engineers (Habiba Kamal, Malak Hany):** Require clean, versioned data streams, robust feature engineering pipelines, and MLOps tools to build, train, and evaluate models (LSTM, Random Forest, XGBoost).

· **MLOps & IT Infrastructure Teams (Jana Mohamed):** Need a scalable architecture, seamless API integration with existing IoT systems, and automated monitoring/retraining pipelines.

· **Documentation & Integration Lead (Sama Saed):** Requires comprehensive system architecture, user guides, and clear visualization dashboards for stakeholder presentations.

2. User Stories & Use Cases

· **Maintenance Manager:** wants to receive an automated alert when a machine's failure probability exceeds 80% within the next 7 days, so I can order parts and schedule maintenance during planned downtime.

· **Technician:** wants to view time-series visualizations of vibration and temperature anomalies leading up to a predicted failure, so I can quickly diagnose the root cause.

· **ML Engineer:** wants to track model performance and trigger automated retraining when data drift is detected, so the system remains accurate as equipment ages.

· **Real-Time Monitoring:** The system continuously ingests IoT sensor data (vibration, temperature, pressure, humidity) and displays equipment health on a centralized dashboard.

· **Proactive Maintenance Trigger:** The API receives sensor data, runs it through the ML model, and triggers an automated maintenance ticket if a failure is predicted.

3. Functional Requirements

· **Sensor Data Collection:** The system shall collect sensor data from public datasets or simulated data, including vibration, temperature, pressure, and humidity.

· **Data Preprocessing:** The system shall identify and handle missing values, outliers, anomalies, and imbalanced data.

· **Feature Engineering and Analysis:** The system shall extract useful features, analyze time-series patterns, and apply PCA to identify important failure indicators.

· **Predictive Model Development:** The system shall train and compare LSTM, Random Forest, and XGBoost models to predict equipment failures.

· **Model Evaluation and Optimization:** The system shall evaluate models using Precision, Recall, F1-score, and ROC-AUC, and optimize them through hyperparameter tuning.

· **Data Visualization:** The system shall generate charts and heatmaps to display sensor trends, anomalies, degradation patterns, and model results.

· **API Deployment and Integration:** The system shall deploy the selected model using FastAPI or Flask and support integration with industrial IoT systems and failure alerts.

· **MLOps and Monitoring:** The system shall track model versions, monitor performance and data drift, and support automated retraining using new data.

4. Non-functional Requirements

· **Performance:** The system should generate predictions within an acceptable response time.

· **Reliability:** The system should provide consistent predictions and handle invalid or missing data appropriately.

· **Scalability:** The system should support increasing data volumes and additional equipment.

· **Security:** The system should protect sensor data and API access from unauthorized users.

· **Usability:** The system should present predictions and analysis results through clear visualizations and reports.

· **Maintainability:** The system should have organized, modular code that is easy to update and debug.

· **Reproducibility:** The system should maintain dataset versions, model versions, and experiment records to reproduce results.

· **Efficiency and Availability:** The system should use computational resources efficiently and keep the prediction API accessible when needed.